

进阶-性能优化、工程化

1. 图片懒加载

提高网页加载速度
减少后台服务器压力
提升用户体验

将图片存到 data-xxx 属性上
判断图是否在可视区域
在视设置图片 src
绑定 scroll 监听事件

2. 节流和防抖 手写!!

① 节流

限制函数的执行频率, 避免过多的重复操作, 提升页面的响应速度

⇒ 函数在 n 秒内只执行一次, 多次触发, 则忽略执行

应用场景: 拖拽场景、scroll 场景、窗口大小调整

② 防抖

将多次高频触发的函数执行合并成一次, 并在规定的時間间隔后执行一次

⇒ 函数在 n 秒后再执行, 如 n 秒内被触发, 重新计时, 保证最后一次触发 n 秒后才执行

应用场景: 输入框搜索、表单提交按钮、文本框保存

3. 性能优化方式有哪些?

① HTML、CSS

减少 DOM 数量

用异步和延迟加载 JS 文件, 避免 JS 文件阻塞页面渲染

压缩 HTML、CSS 代码体积

减少使用复杂 CSS 选择器

避免使用 CSS 表达式在 JS 代码中

使用 CSS 渲染合成层如 transform、opacity、will-change

动画使用 CSS3 过渡, 减少动画复杂度, 还可以使用硬件加速

② JS

减少 DOM 操作数量

避免使用 with 语句、eval 函数, 避免引擎难以优化

- 尽量使用原生方法, 执行效率高。
- 将 js 文件放到文件页面底部, 避免阻塞页面渲染
- 使用事件委托, 减少事件绑定次数。
- 合理使用缓存, 避免重复请求数据

③ Vue

- 合理使用 watch 和 computed, 数据变化就会执行, 避免使用太多, 减少不必要的开销
- 合理使用组件, 提高代码可维护性的同事也会降低代码组件的耦合性
- 使用路由懒加载, 在需要的时候才会进行加载, 避免一次性加载太多路由, 导致页面阻塞
- 使用 Vuex 缓存数据
- 合理使用 mixins, 抽离公共代码封装成模块, 避免重复代码。
- 合理使用 v-if、v-show

- v-for 不要和v-if一起使用，v-for的优先级会比v-if高
- v-for中不要用index做key，要保证key的唯一性
- 使用异步组件，避免一次性加载太多组件
- 避免使用v-html，存在安全风险和性能问题，可以使用v-text
- 使用keep-alive缓存组件，避免组件重复加载

④ Webpack

- 代码切割，使用code splitting将代码进行分割，避免将所有代码打包到一个文件，减少响应体积。
- 按需加载代码，在使用使用的时候加载代码。
- 压缩代码体积，可以减小代码体积
- 优化静态资源，使用字体图标、雪碧图、webp格式的图片、svg图标等
- 使用Tree Shaking 删除未被引用的代码
- 开启gzip压缩
- 静态资源使用CDN加载，减少服务器压力

⑤ 网络优化

- 使用HTTP/2
- 减少、合并HTTP请求，通过合并CSS、JS文件、精灵图等方式减少请求数量。
- 压缩文件，开启nginx，Gzip对静态资源压缩
- 使用HTTP缓存，如强缓存、协商缓存
- 使用CDN，将网站资源分布到各地服务器上，减少访问延迟

4. vite 与 webpack 的区别

	Webpack	Vite
核心原理	打包优先 (先打包再启动服务器)	原生ESM, 启动服务器时无需打包
冷启动速度	慢	快
热更新	慢, 修改后需重新编译模块	快, 基于ESM精准更新单个模块
生产构建	较慢, 全量打包优化	较快, 用Rollup打包, 常配合esbuild
生态插件	成熟	快速增长中



为什么快?

① 开发环境免打包

② 底层环境: 用esbuild预构建依赖
↳ 用Go语言写的, 比用JS编写的Webpack快